

Claude Fit Logger

Security & Credential Rotation Guide

1 Credential Inventory

Credential	Where stored	Scope of compromise if leaked
Google OAuth Client Secret	Render env, local notes	Google account nutrition data
Google Refresh Token	Render env, local notes	Google account nutrition data
Neon DB Password	Render env, local notes	Entry log (food history only)
USDA API Key	Render env	Low — public data, rate limit only
MCP API Key	Render env, Claude connector	Access to your MCP server's tools

2 General Hygiene Rules

- Never paste raw credential values into a chat interface, screenshot, or commit message. Treat any value that appears in such a context as compromised, whether or not it was actually misused.
- `.gitignore` must exclude all credential-bearing files *before* their first commit — adding it afterward does not remove them from git history.
- Prefer environment variables (Render) over files for anything deployed; keep local credential notes in a file excluded from version control.

3 Rotation Procedures

3.1 Google OAuth Client Secret + Refresh Token

These must be rotated together — a refresh token is only valid against the client secret it was issued with.

1. Google Cloud Console → *Clients* → select the client → **Add secret**.
2. Download the new `client_secret.json` for that specific secret (do not reuse an old downloaded copy).
3. Run the refresh-token script locally against the new file to mint a new refresh token.
4. Update `GOOGLE_CLIENT_SECRET` and `GOOGLE_REFRESH_TOKEN` on Render.
5. Delete the old secret from the Clients page.

3.2 Neon Database Password

1. Neon console → project → *Postgres database* → role settings → **Reset password**.
2. Reassemble the full connection string with the new password (host/database name are unchanged).

3. Update `DATABASE_URL` on Render.
4. Verify locally: `psycpg.connect(new_string)` before relying on it in production.

3.3 USDA API Key

1. Re-run the FoodData Central signup form with the same email to receive a new key (or contact USDA support to revoke the old one).
2. Update `USDA_API_KEY` on Render.

3.4 MCP API Key

1. Generate a new value: `python -c "import secrets; print(secrets.token_hex(32))"`
2. Update `MCP_API_KEY` on Render.
3. Reconnect the Claude connector (it will re-run the OAuth shim handshake, which issues this key as the underlying bearer token).

4 Threat Model Notes

- This system is single-user by design. The OAuth shim (`oauth.py`) auto-approves every authorization request with no real login check — this is acceptable only because the server's URL and API key are not published, and because the blast radius of a leak is limited to one person's nutrition log, not a multi-tenant system.
- Do not extend this system to other users without adding real authentication to `oauth.py`'s authorize step.